

BookStore

Full Stack Development with MERN

Project Documentation

A three-portal online bookstore — Customer, Seller, and Admin — built on MongoDB, Express, React, and Node.js.

1. Introduction

Project Title

BookStore — A Full Stack MERN E-Commerce Platform for Books

Team Members

The project was built by a five-member team, with responsibilities split across planning, frontend, backend, and database/testing work.

S.No	Team Member	Role	Responsibilities
1	Sana Hema Naga Satya Bhavani	Team Lead	Project planning, team coordination, backend integration, final project management
2	Bala Sai Siraparapu	Full Stack Developer	Frontend development, backend API integration, database connectivity, testing and documentation
3	Appanapalli Sunitha	Frontend Developer	Designing React components, creating UI pages, improving user experience
4	Addanki Veera Venkata Sai Ganesh	Backend Developer	Developing REST APIs, server-side logic, authentication implementation
5	Sarup Bormon	Database & Testing Member	MongoDB database management, API testing, bug fixing and project support

2. Project Overview

Purpose

BookStore is an online bookstore application that goes beyond a simple single-sided shopping site. It is a small marketplace with three distinct portals sharing one catalog and one database: a customer-facing storefront, a seller dashboard for vendors to list and manage their own books, and an admin console to oversee the entire platform. The goal was to model a realistic e-commerce workflow — registration and approval, cataloguing, cart and checkout, order fulfillment, and platform administration — end to end.

Key Features

Customer (User) side

- Browse, search, and filter the book catalog by category and language
- Wishlist with a redesigned empty state and populated grid view
- Cart and checkout with address and payment method selection

- Order history with live status tracking, and cancellation that captures a reason from the customer
- Editable profile with photo upload, persisted to the account
- Personalized home page with a reading shelf and recommendations

Seller side

- Registration with a mandatory admin-approval step before selling
- Dashboard with sales statistics (listings, orders, units sold, revenue)
- Add Book page with a live cover preview and automatic cover generation when no image is uploaded
- Product management (edit price/discount/stock, remove listings)
- Order fulfillment view with per-item status updates (Pending → Processing → Shipped → Delivered)

Admin side

- Platform-wide dashboard (users, sellers, books, orders, sales)
- User management (activate/suspend/delete)
- Seller management (approve/block/delete pending or existing sellers)
- Book management across all sellers
- Order oversight across the whole platform
- Admin creation restricted to existing admins only — no public admin sign-up

Cross-cutting features

- One JWT authentication scheme shared by all three roles
- Confirmation prompts before destructive actions (logout, order cancellation)
- Consistent, distinctive visual design (custom CSS theme, no UI framework)

3. Architecture

Frontend — React

The client is a single React application (Create React App) split into three route trees, one per portal, using React Router. State that needs to be shared across pages — the logged-in actor (user/seller/admin) and the shopping cart — is managed through React Context (AuthContext and CartContext) rather than a separate state-management library. Components are functional, using hooks (useState, useEffect, useCallback) throughout. Axios wraps all calls to the backend REST API, and react-toastify handles inline success/error notifications.

Reusable UI pieces — BookCard, BookCover (with automatic cover generation), ConfirmModal, EmptyState, Navbar variants for each portal — live under Components/, Seller/, and Admin/ alongside the pages that use them, keeping each portal's pages self-contained while sharing common building blocks.

Backend — Node.js and Express

The server exposes a single REST API under /api, organized by resource (users, sellers, admin, books, authors, categories, cart, wishlist, orders, reviews). Cross-cutting concerns — authentication, rate limiting on login/register routes, security headers (helmet), and request logging (morgan) — are applied as Express middleware in server.js.

A single authentication middleware (verifyToken) verifies the JWT on every protected request and resolves it against the correct Mongoose model (User, Seller, or Admin) based on an actorType claim embedded in the token. Role-specific guards (requireRole, requireApprovedSeller) then restrict individual routes.

Database — MongoDB

Data is modeled with Mongoose across the following main collections:

- User, Seller, Admin — one collection per actor type, each with its own auth fields
- Book — the catalog, referencing Author and Category as ref-arrays (many-to-many)
- Inventory — quantity/location/condition batches per book; Book.stock is kept as a synced aggregate for fast catalog/cart reads
- Cart, Wishlist — one document per user, referencing books
- Order — line items each carry their own seller reference and fulfillmentStatus, so a single order spanning multiple sellers' books can be fulfilled independently by each seller; a cancelReason field records why a customer cancelled
- Review — rating and comment per user/book pair
- Interaction — tracks a user's reading status/progress per book, separate from Review

4. Setup Instructions

Prerequisites

- Node.js (v18 or later recommended) and npm
- MongoDB — either a local instance (mongod) or a free MongoDB Atlas cluster
- Git (to clone the repository)

Installation

1. Clone the repository and move into the project folder:

```
git clone <repository-url>
cd bookstore-mern
```

2. Install dependencies for the root, server, and client in one step:

```
npm run install-all
```

(This runs npm install at the root, then inside server/, then inside client/.)

3. Configure environment variables. Copy the example file and fill in real values:

```
cd server
cp .env.example .env
```

Variable	Purpose
PORT	Port the backend listens on (default 5000)
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret key used to sign/verify JWTs
JWT_EXPIRE	Token lifetime, e.g. 7d

CLIENT_URL	Frontend origin, used for CORS
ADMIN_EMAIL / ADMIN_PASSWORD	Used only by the seed script to create the first admin

4. (Optional but recommended) Seed the database with sample users, sellers, an admin, categories, and books:

```
npm run seed
```

5. Folder Structure

Client (React frontend)

```
client/src/
  Admin/      Ahome, Alogin, Anavbar, Asignup, Seller.js, Users.js, items.js, Orders.js
  Seller/     Shome, Slogin, Snavbar, Ssignup, Addbook, MyProducts, Book, Orders, Profile
  User/       Uhome, Login, Signup, Products, Uitem, Cart, Checkout, MyOrders,
              OrderItem, CancelOrderModal, Wishlist, Profile, ForgotPassword
  Components/ Home, Navbar, Footer, BookCard, BookCover, ConfirmModal,
              EmptyState, ProtectedRoute, NotFound
  context/    AuthContext (all three roles), CartContext
  services/   api.js (Axios instance, base URL, auth header)
  utils/      coverGenerator.js (automatic book-cover generation)
  App.js      Route definitions for all three portals
  App.css     Shared design system (theme colors, typography, animations)
```

Server (Node.js backend)

```
server/
  config/     connect.js (MongoDB connection), seed.js (demo data)
  controllers/ UsersController, SellerControllers, AdminControllers,
              bookController, authorController, categoryController,
              cartController, wishlistController, orderController, reviewController
  middleware/ authMiddleware.js (JWT verification + role guards), upload.js (Multer)
  models/     Users/User.js, Seller/Seller.js, Admin/Admin.js, Book.js, Author.js,
              Category.js, Inventory.js, Cart.js, Wishlist.js, Order.js,
              Review.js, Interaction.js
  routes/     userRoutes, sellerRoutes, adminRoutes, bookRoutes, authorRoutes,
              categoryRoutes, cartRoutes, wishlistRoutes, orderRoutes, reviewRoutes
  uploads/    Uploaded cover images and profile pictures (served as static files)
  server.js   Express app setup, middleware, route mounting
```

6. Running the Application

The backend and frontend run as two separate processes.

Option A — One command from the project root

```
npm run dev
```

This uses concurrently to start both the backend (nodemon server.js) and the frontend (react-scripts start) in the same terminal.

Option B — Two terminals

Backend:

```
cd server
npm start
# or: npm run dev (auto-restarts on file changes via nodemon)
```

Frontend:

```
cd client
npm start
```

By default, the backend runs on <http://localhost:5000> and the frontend on <http://localhost:3000>, with the client proxying API calls to the backend as configured in `client/package.json`.

7. API Documentation

All endpoints are mounted under `/api`. Protected routes require an Authorization: Bearer <token> header; the "Access" column below shows which role(s) the route is restricted to.

User endpoints (`/api/users`)

Method	Endpoint	Access	Description
POST	<code>/api/users/register</code>	Public	Register a new customer account
POST	<code>/api/users/login</code>	Public	Log in and receive a JWT token
POST	<code>/api/users/forgot-password</code>	Public	Request a password-reset token by email
POST	<code>/api/users/reset-password/:token</code>	Public	Set a new password using a reset token
GET	<code>/api/users/me</code>	User	Get the logged-in user's own profile
PUT	<code>/api/users/profile</code>	User	Update profile details / upload profile picture
GET	<code>/api/users/order-history</code>	User	Get the user's past orders
GET	<code>/api/users/interactions</code>	User	Get the user's reading shelf / book interactions
PUT	<code>/api/users/interactions/:bookId</code>	User	Set reading status for a book (want to read / reading / completed)

Seller endpoints (`/api/sellers`)

Method	Endpoint	Access	Description
POST	/api/sellers/register	Public	Register a new seller account (starts as "pending")
POST	/api/sellers/login	Public	Seller login, receives a JWT token
GET	/api/sellers/me	Seller	Get the logged-in seller's own profile
PUT	/api/sellers/profile	Seller	Update business details / upload profile picture
GET	/api/sellers/stats	Seller (approved)	Dashboard stats: listings, orders, units sold, sales
GET	/api/sellers/my-products	Seller (approved)	List all books this seller has listed
POST	/api/sellers/books	Seller (approved)	Add a new book (auto-generates a cover if none uploaded)
PUT	/api/sellers/books/:id	Seller (approved)	Edit one of the seller's own books
DELETE	/api/sellers/books/:id	Seller (approved)	Remove one of the seller's own books
GET	/api/sellers/orders	Seller (approved)	Get orders containing this seller's books
PUT	/api/sellers/orders/:orderId/items/:bookId/status	Seller (approved)	Update fulfillment status of one order line item

Admin endpoints (/api/admin)

Method	Endpoint	Access	Description
POST	/api/admin/login	Public	Admin login, receives a JWT token
GET	/api/admin/me	Admin	Get the logged-in admin's own profile
POST	/api/admin/create	Admin	Create another admin account (no public admin sign-up)
GET	/api/admin/stats	Admin	Platform-wide dashboard statistics
GET	/api/admin/users	Admin	List all registered customers
PUT	/api/admin/users/:id/status	Admin	Activate or suspend a customer account
DELETE	/api/admin/users/:id	Admin	Delete a customer account
GET	/api/admin/sellers	Admin	List all sellers, including pending ones
PUT	/api/admin/sellers/:id/approve	Admin	Approve a pending seller
PUT	/api/admin/sellers/:id/block	Admin	Block a seller account
DELETE	/api/admin/sellers/:id	Admin	Delete a seller account

Method	Endpoint	Access	Description
GET	/api/admin/books	Admin	List every book on the platform
POST	/api/admin/books	Admin	Add a book on behalf of the platform
PUT	/api/admin/books/:id	Admin	Edit any book's details or status
DELETE	/api/admin/books/:id	Admin	Remove any book from the platform

Catalog endpoints (/api/books, /api/authors, /api/categories)

Method	Endpoint	Access	Description
GET	/api/books	Public	Browse / search / filter the book catalog (paginated)
GET	/api/books/:id	Public	Get full details for one book
GET	/api/authors	Public	List all authors
GET	/api/categories	Public	List all categories
POST	/api/categories	Admin	Create a category
PUT	/api/categories/:id	Admin	Update a category
DELETE	/api/categories/:id	Admin	Delete a category

Cart & Wishlist endpoints (/api/cart, /api/wishlist)

Method	Endpoint	Access	Description
GET	/api/cart	User	Get the current cart
POST	/api/cart/add	User	Add a book to the cart
PUT	/api/cart/update	User	Change the quantity of a cart item
DELETE	/api/cart/remove/:bookId	User	Remove a book from the cart
POST	/api/cart/coupon	User	Apply a discount coupon
DELETE	/api/cart/clear	User	Empty the cart
GET	/api/wishlist	User	Get the current wishlist
POST	/api/wishlist/add	User	Save a book to the wishlist
DELETE	/api/wishlist/remove/:bookId	User	Remove a book from the wishlist

Order & Review endpoints (/api/orders, /api/reviews)

Method	Endpoint	Access	Description
POST	/api/orders	User	Place an order from the current cart

Method	Endpoint	Access	Description
GET	/api/orders/my-orders	User	Get the logged-in user's orders
GET	/api/orders	Admin	Get every order on the platform
GET	/api/orders/:id	User / Admin / Seller	Get one order (access checked per-role in the controller)
PUT	/api/orders/:id/cancel	User	Cancel an order and record the customer's reason
PUT	/api/orders/:id/status	Admin	Update an order's overall status
GET	/api/reviews	Admin	List every review submitted
POST	/api/reviews	User	Submit a review for a book
PUT	/api/reviews/:id	User	Edit the user's own review
DELETE	/api/reviews/:id	User / Admin	Delete a review

Example request / response

POST /api/orders/:id/cancel (User)

Request body:

```
{
  "reason": "Delivery is taking too long"
}
```

Response (200):

```
{
  "success": true,
  "order": {
    "_id": "665f1c...",
    "status": "Cancelled",
    "cancelReason": "Delivery is taking too long",
    "...": "..."
  }
}
```

8. Authentication

Authentication is handled with JSON Web Tokens (JWT), shared across all three actor types rather than three separate systems.

- On login/register, the server signs a token containing { id, actorType }, where actorType is one of "user", "seller", or "admin".
- The client stores the token (and the "actor" object) and attaches it to every subsequent request as an Authorization: Bearer <token> header.

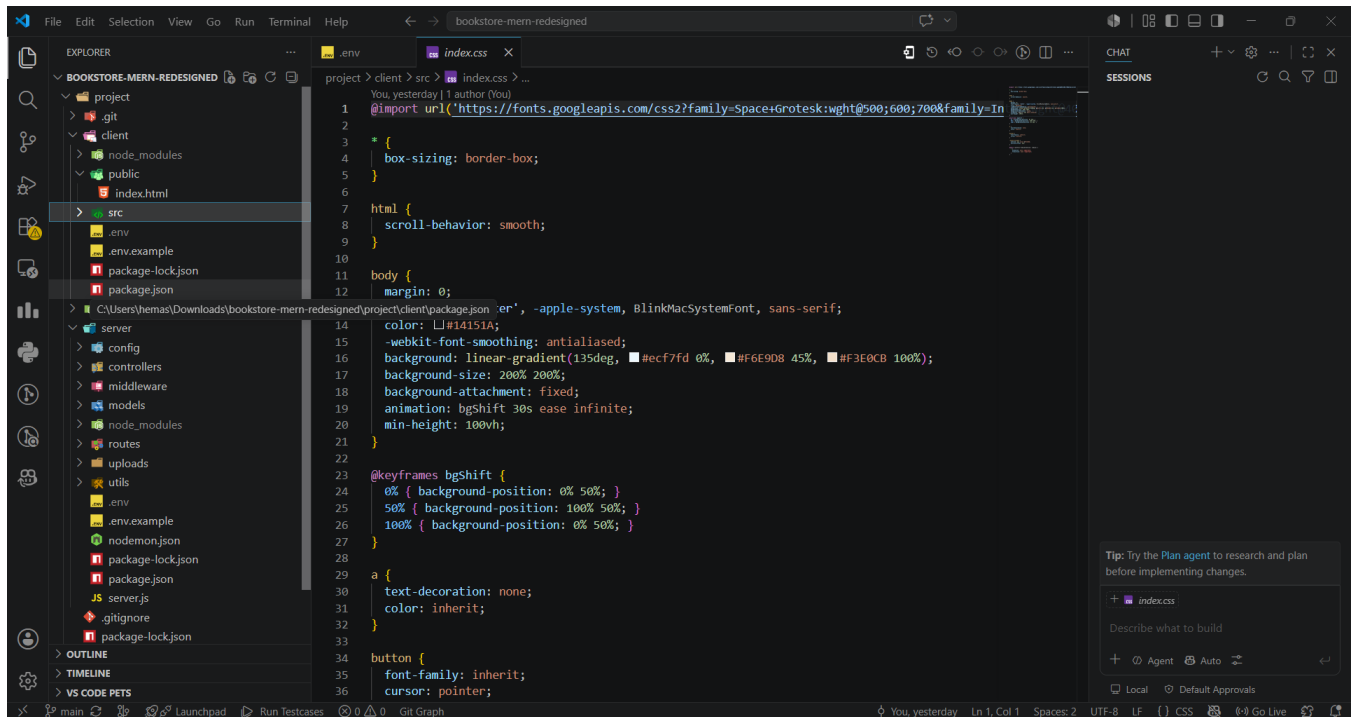
- A single Express middleware, `verifyToken`, decodes the token, looks the account up in the matching Mongoose model (User, Seller, or Admin), and attaches it to the request as `req.actor / req.actorType`.
- Passwords are hashed with `bcryptjs` before being stored; they are never returned in API responses.
- Role-based guards restrict routes further: `requireRole('admin')` for admin-only routes, and `requireApprovedSeller` for seller routes that additionally require the seller's account to be approved (not pending or blocked) — enforced on the server, not just hidden in the UI.
- Forgot-password flow issues a short-lived reset token by email, which must be presented to set a new password.
- Login, register, and admin-login routes are additionally rate-limited to reduce brute-force risk.

9. User Interface

The interface uses a custom design system rather than a component library: a violet/indigo and gold color palette, 'Space Grotesk' for headings, glassmorphism cards, gradient hero sections, and small CSS animations (fade-ins, floating accents) used consistently across all three portals.

Key screens to capture for a walkthrough :

- Home page, including the redesigned newsletter subscribe section
- Wishlist — both the populated grid and the animated empty state
- My Orders — order history with status tracking and the cancel-with-reason modal
- Seller dashboard — pending-approval, blocked, and active states
- Add Book page — live cover preview and the automatically generated cover
- Order Fulfillment — card-based view with per-item status controls
- Admin dashboard, Seller approval screen, and Create Admin page



10. Testing

Testing was primarily manual and functional, covering each user flow end-to-end across all three portals:

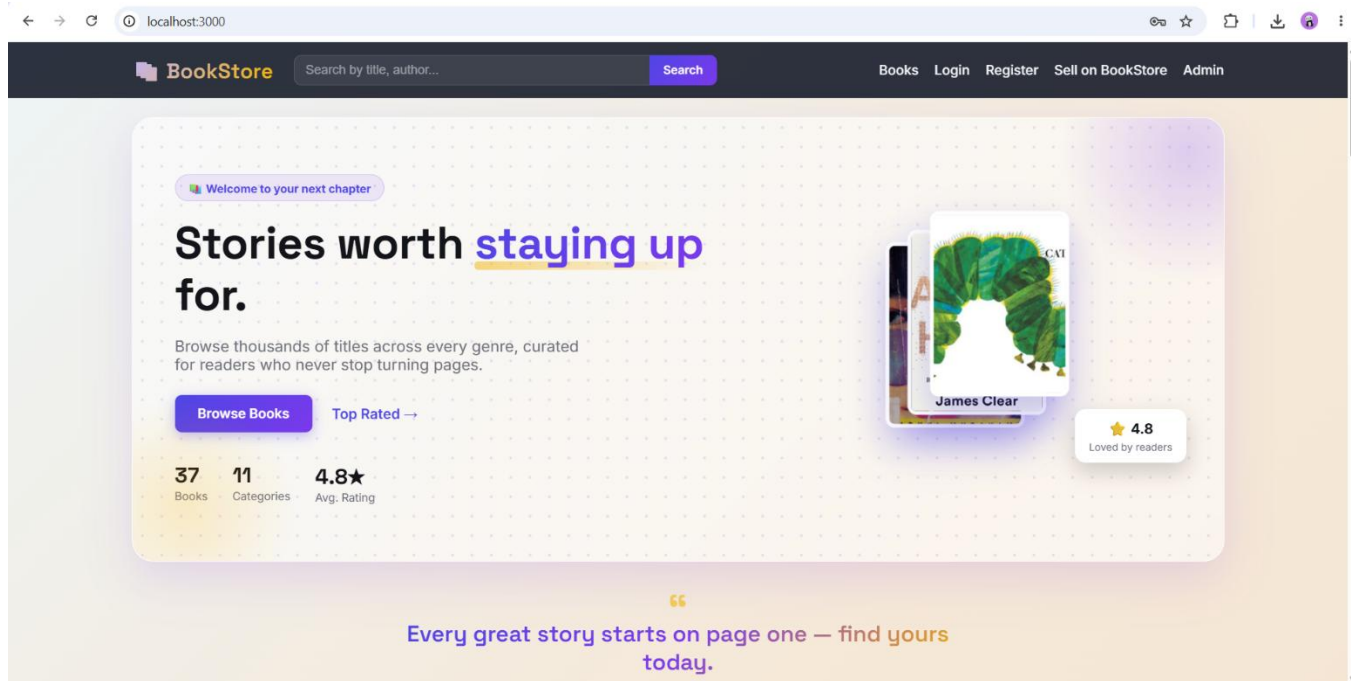
- Manual functional testing of registration/login for all three roles, including the seller pending-approval gate
- Manual testing of the full purchase flow: browse → cart → checkout → order → cancellation
- API testing of individual endpoints (request/response shape, status codes, auth enforcement) using tools such as Postman/Thunder Client during development
- Cross-checking that role guards actually block unauthorized access at the API level, not just in the UI

The client is scaffolded with Create React App, which includes Jest and React Testing Library out of the box (npm test in client/); no automated test suite has been written yet — see Known Issues.

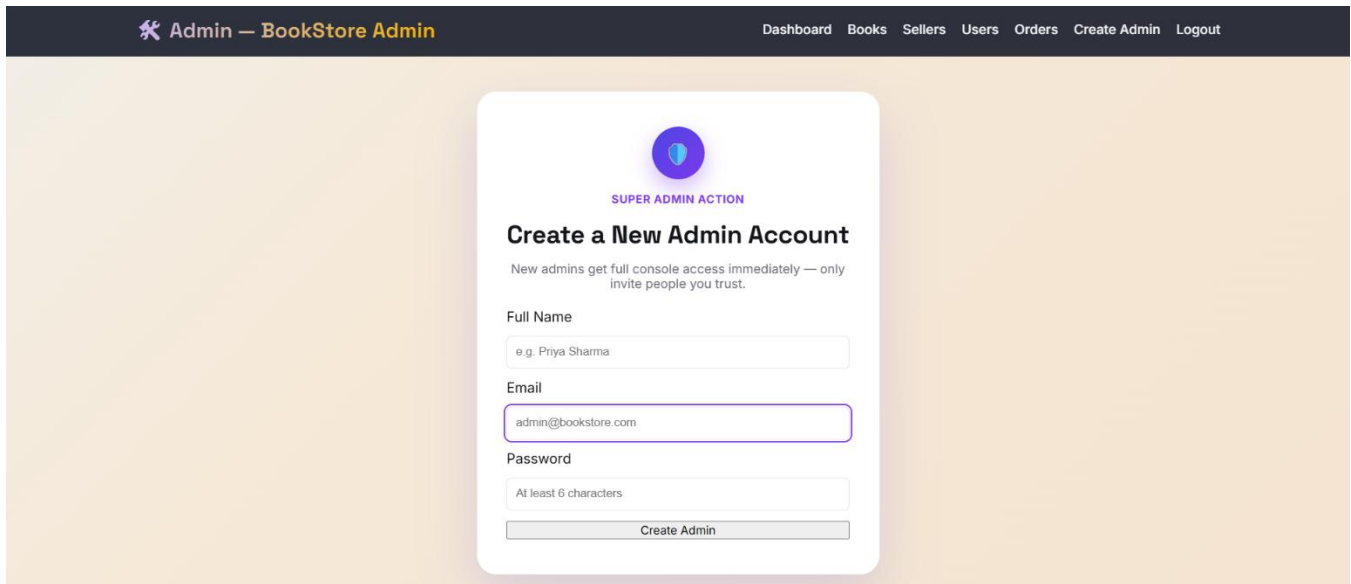
11. Screenshots or Demo

A recorded walkthrough of the application attached here,

HOME PAGE:



ADMIN REGISTRATION :



Admin — BookStore Admin

Dashboard Books Sellers Users Orders Create Admin Logout

SUPER ADMIN ACTION

Create a New Admin Account

New admins get full console access immediately — only invite people you trust.

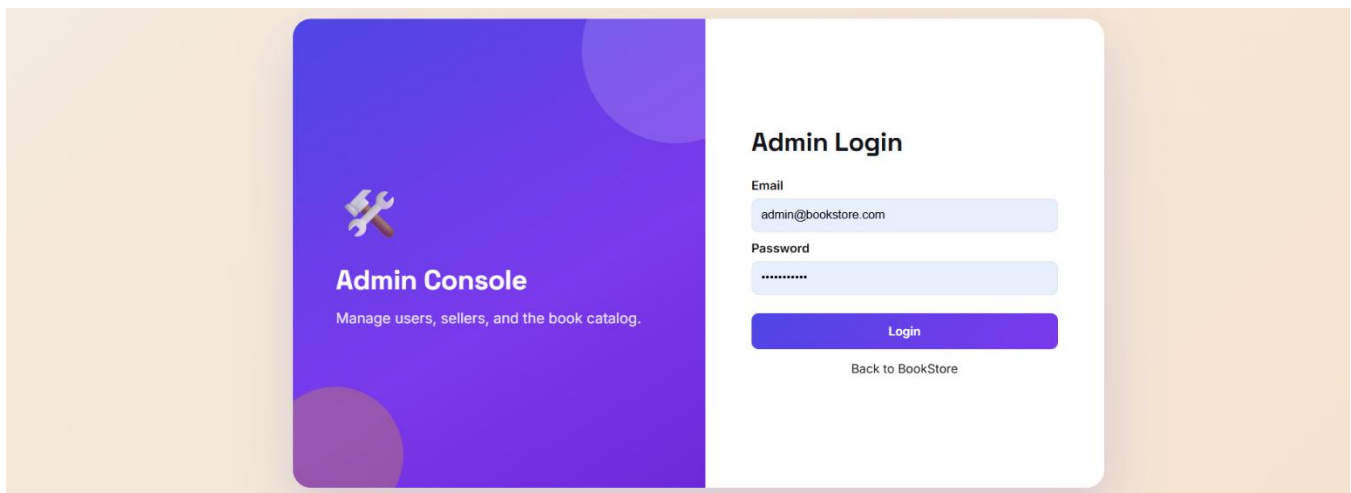
Full Name
e.g. Priya Sharma

Email
admin@bookstore.com

Password
At least 6 characters

Create Admin

ADMIN LOGIN:



Admin Console
Manage users, sellers, and the book catalog.

Admin Login

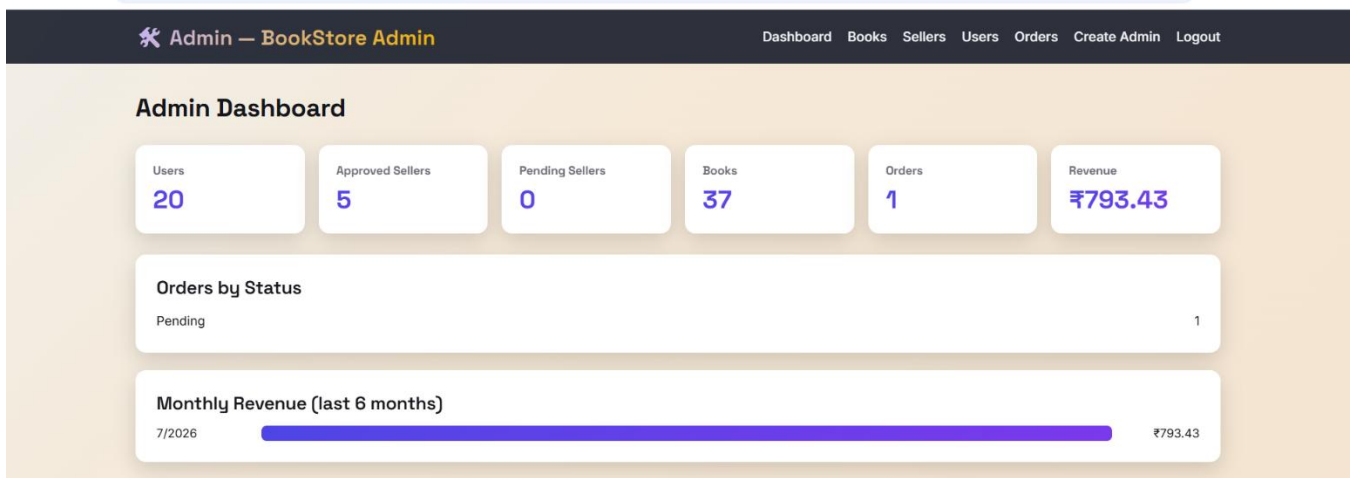
Email
admin@bookstore.com

Password

Login

[Back to BookStore](#)

ADMIN HOME:



ADMIN USER'S SECTION:


Admin — BookStore Admin

[Dashboard](#)
[Books](#)
[Sellers](#)
[Users](#)
[Orders](#)
[Create Admin](#)
[Logout](#)


CUSTOMER ACCOUNTS

User Management

Name	Email	Mobile	Status	Joined	Actions
Test User 1	user1@example.com	9000000001	active	7/16/2026	Suspend Delete
Test User 2	user2@example.com	9000000002	active	7/16/2026	Suspend Delete
Test User 3	user3@example.com	9000000003	active	7/16/2026	Suspend Delete
Test User 4	user4@example.com	9000000004	active	7/16/2026	Suspend Delete
Test User 5	user5@example.com	9000000005	active	7/16/2026	Suspend Delete

ORDER VIEW PAGE:


STOREWIDE

Order Management

All

Order	Customer	Items	Total	Status
#551A8E93	Test User 1	1 item(s)	₹793.43	Shipped

SELLER REGISTRATION:



Become a Seller

Reach thousands of readers. Your account will need a quick admin approval before you can list books.

Seller Registration

Business Name

Owner Name

Email

Mobile Number

GST Number (optional)

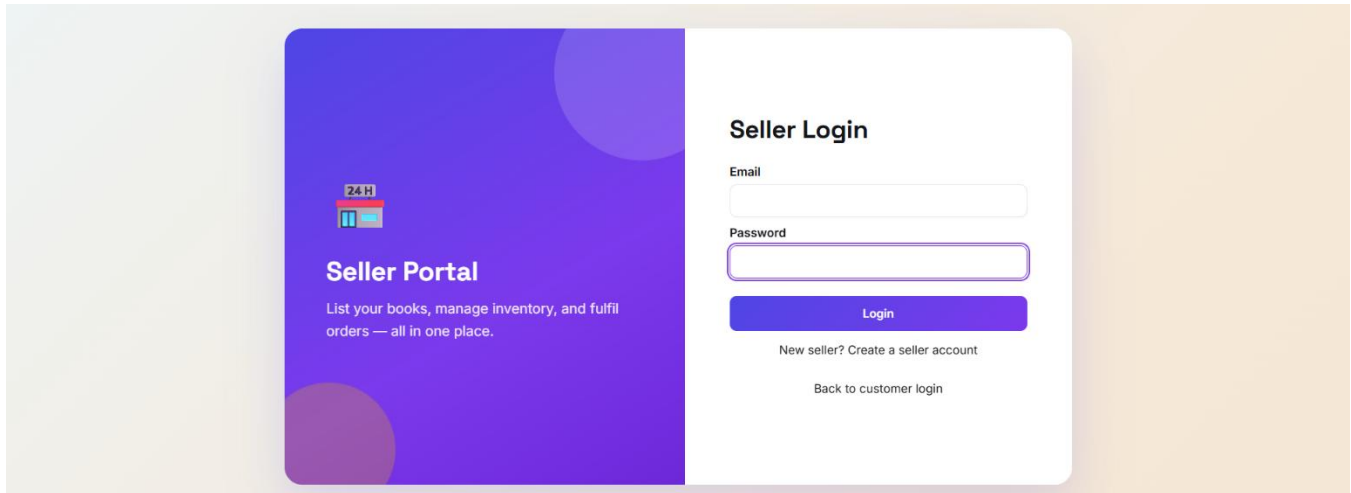
Password

Confirm Password

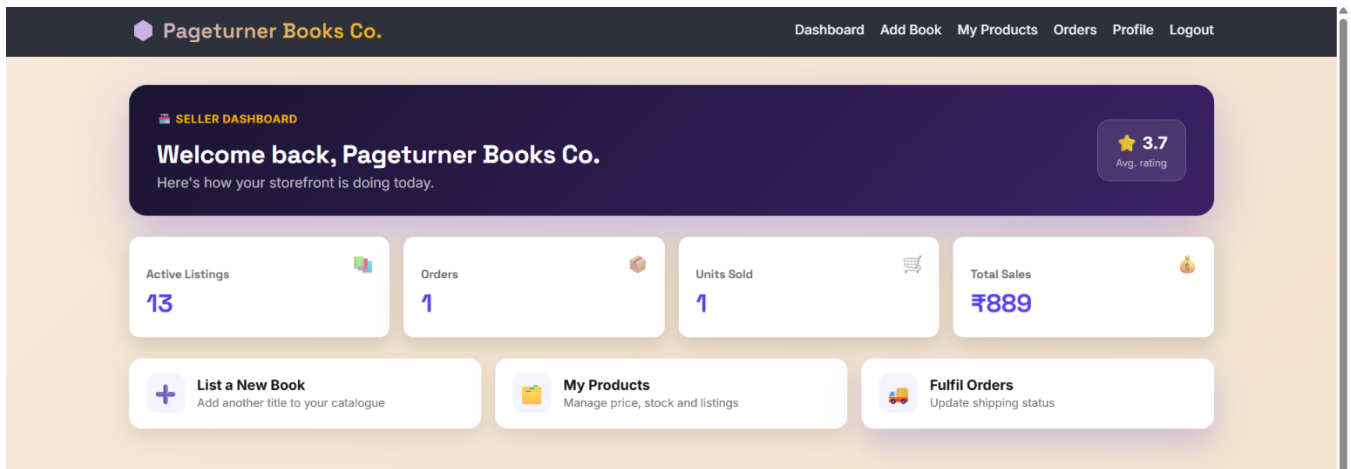
[Register as Seller](#)

Already registered? [Login](#)

SELLER LOGIN:



SELLER HOME:



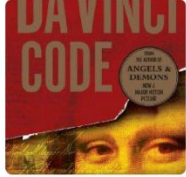
SELLER'S PRODUCTS:

Pageturner Books Co. Dashboard Add Book My Products Orders Profile Logout

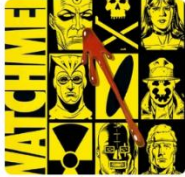
YOUR CATALOGUE

My Products

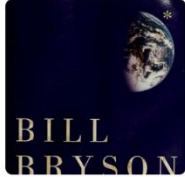
[+ List New Book](#)



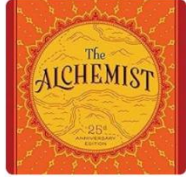
The Da Vinci Code
₹889 30 in stock
Fiction Active
[Edit](#) [Remove](#)



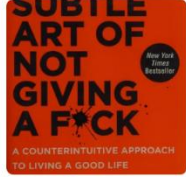
Watchmen
₹167 92 in stock
Comics Active
[Edit](#) [Remove](#)



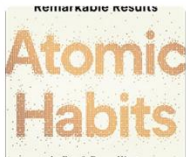
A Short History of Nearly Everything
₹724 34 in stock
Science Active
[Edit](#) [Remove](#)



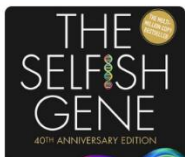
The Alchemist
₹718 55 in stock
Fiction Active
[Edit](#) [Remove](#)



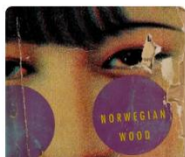
The Subtle Art of Not Giving a F*ck
₹536 30 in stock
Self Help Active
[Edit](#) [Remove](#)



Atomic Habits
An Easy & Proven Way to



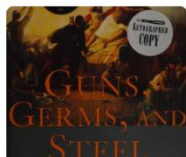
THE SELFISH GENE
40TH ANNIVERSARY EDITION



NORWEGIAN WOOD



STARTUP
How Today's Entrepreneurs Use Continuous Innovation to Create Radically Disruptive Businesses



GUNS, GERMS, AND STEEL
An Illustrated Edition

ADDBOOK PAGE:

Pageturner Books Co. Dashboard Add Book My Products Orders Profile Logout

[/ LIST A BOOK](#)

Add a New Book

Fill in the details on the left — watch the cover come together on the right.

Book Details

Title

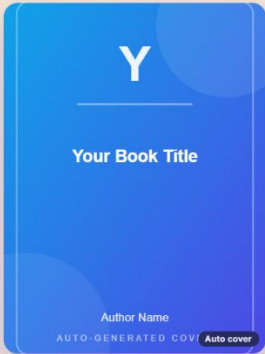
Author(s) — comma separated

Description

Categories

☐ Biography
 ☐ Business
 ☐ Children
 ☐ Comics
 ☐ Fiction
 ☐ History
 ☐ Non Fiction
 ☐ Programming
 ☐ Science
 ☐ Self Help
 ☐ Technology

LIVE AUTO-GENERATED PREVIEW



Your Book Title

Author Name

AUTO-GENERATED COVER [Auto cover](#)

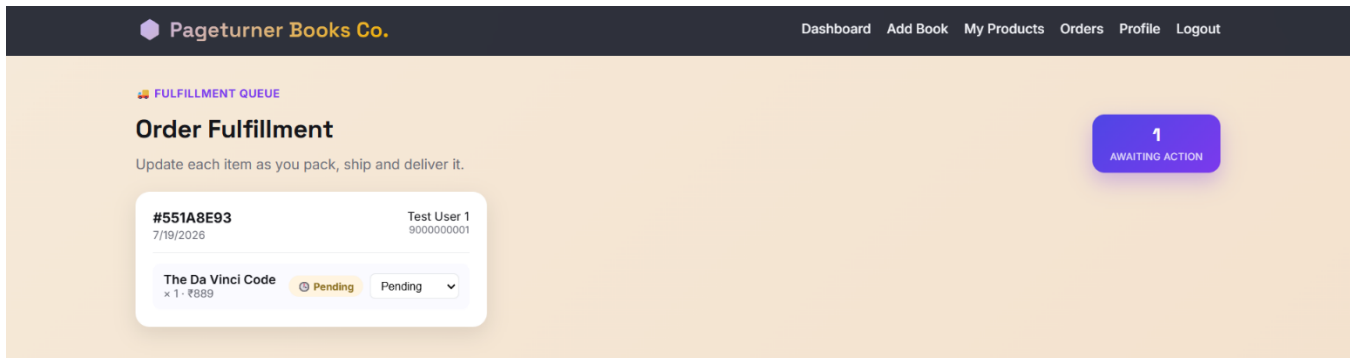
Your Book Title

Author Name

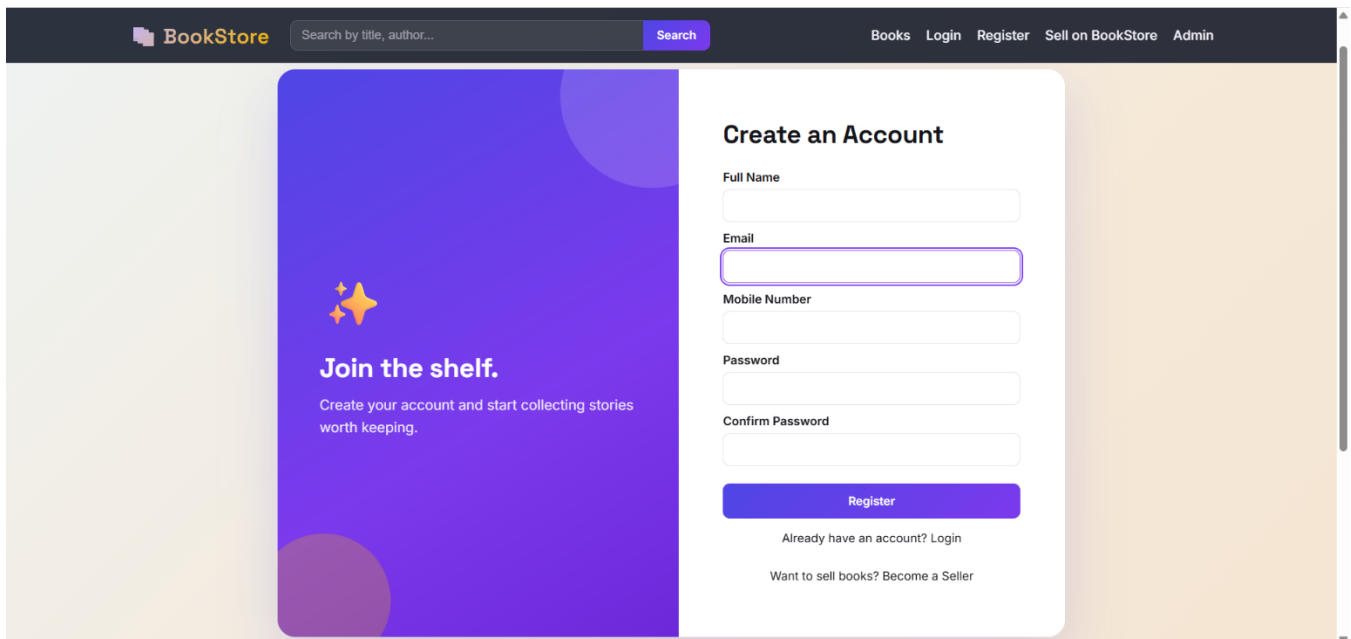
₹0

Pricing & Stock

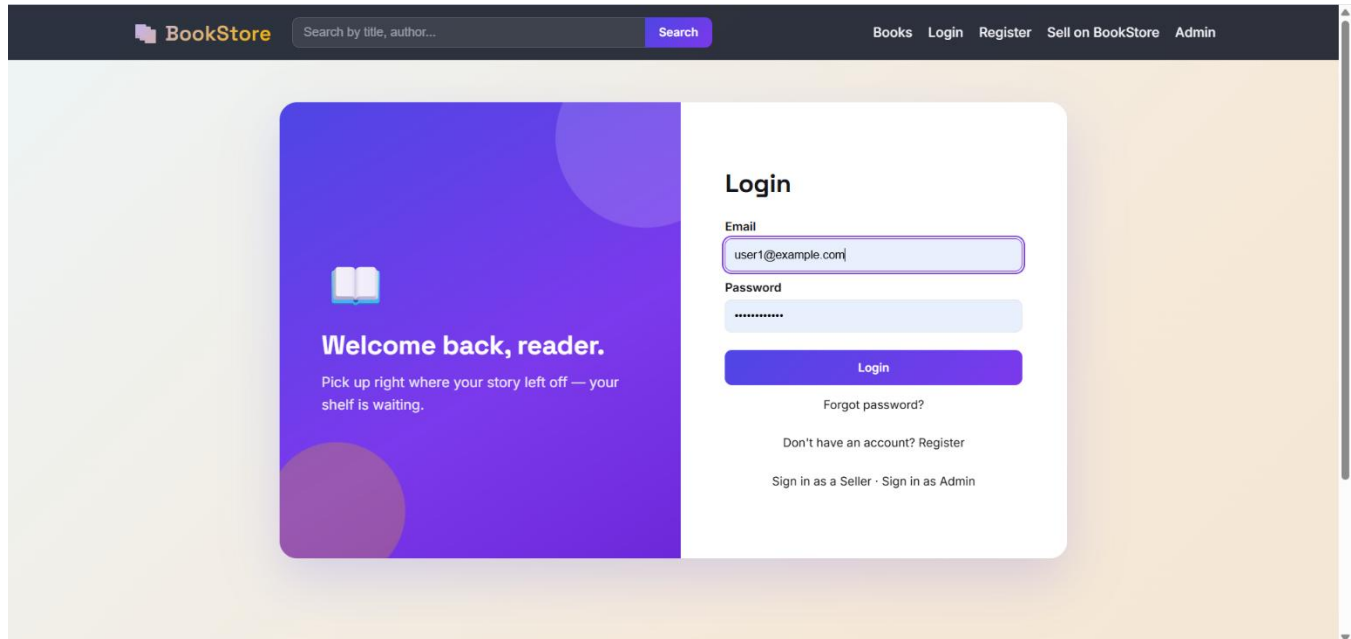
ORDER'S PAGE:



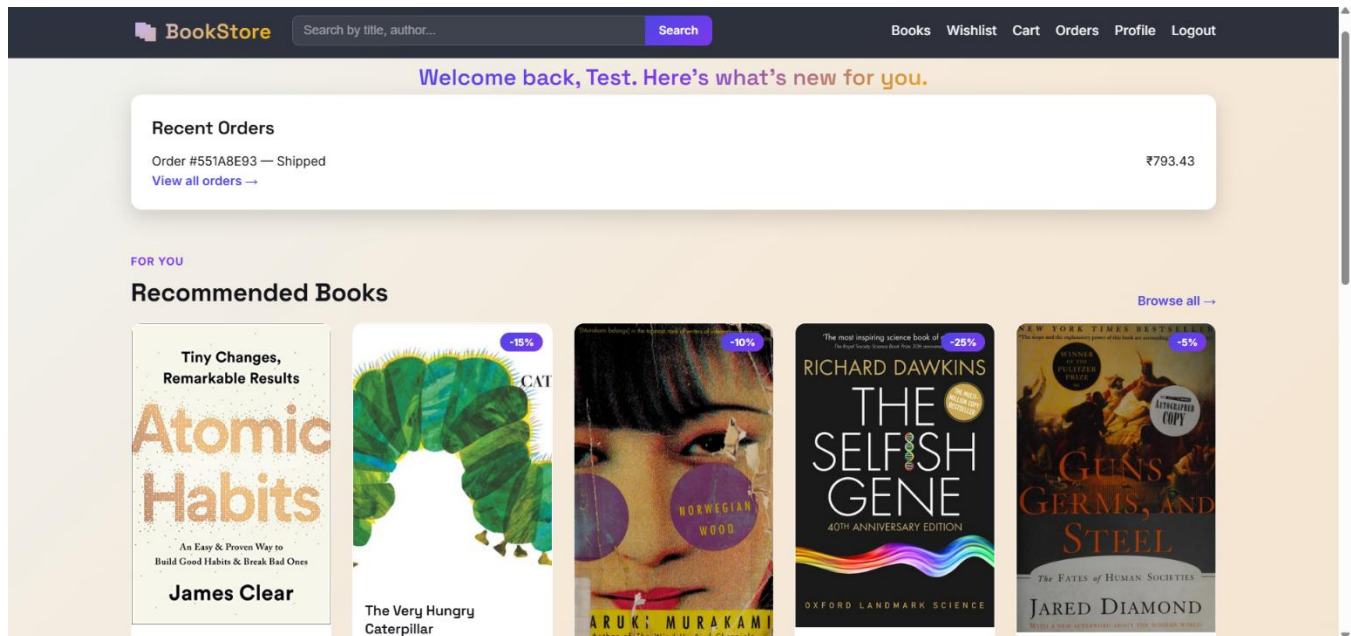
USER REGISTRATION PAGE:



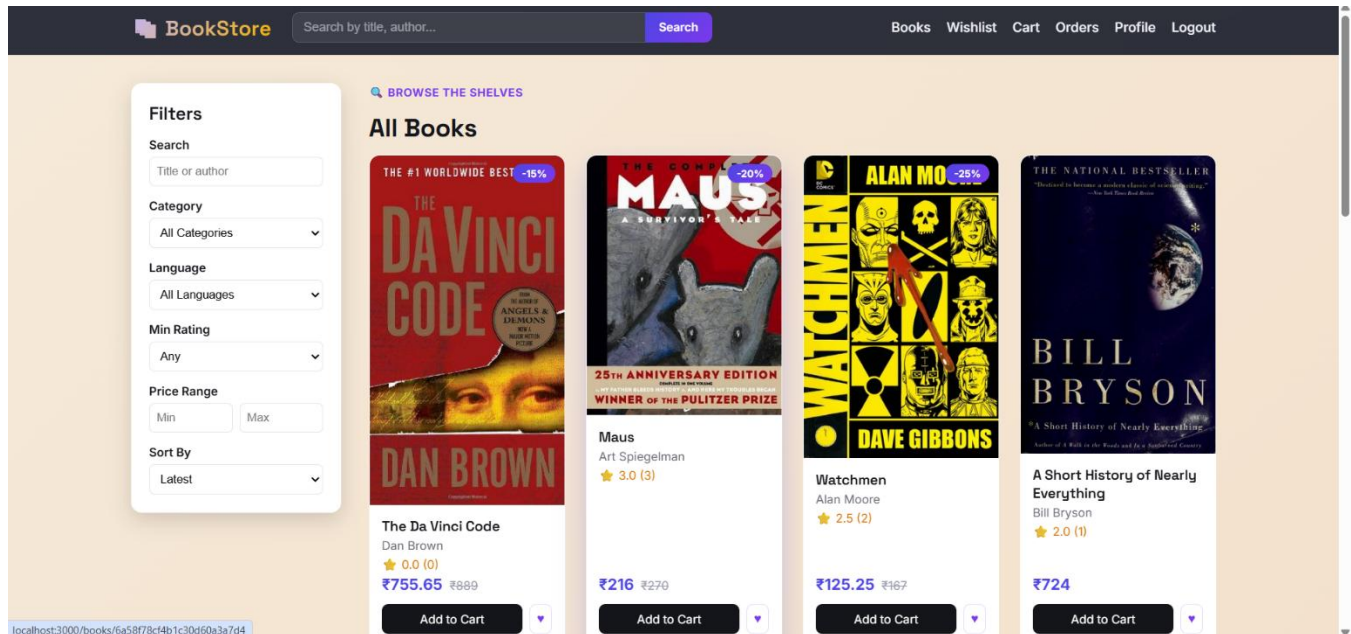
USER LOGIN PAGE:



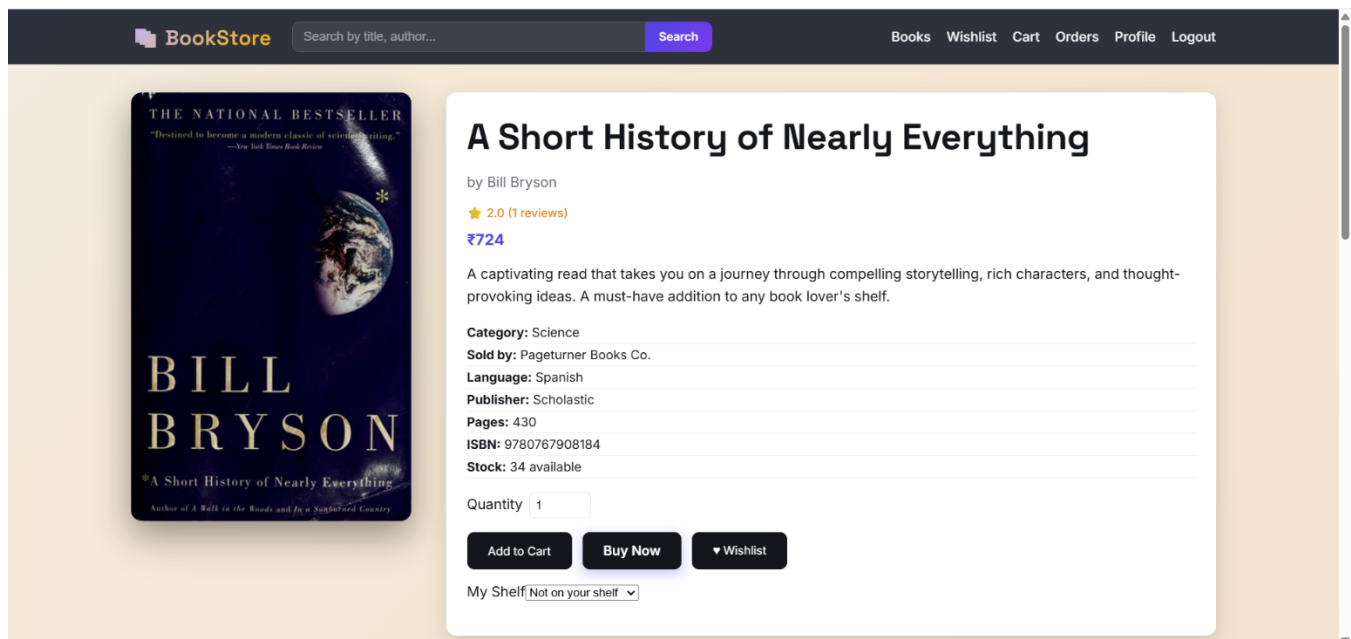
USER HOME PAGE:



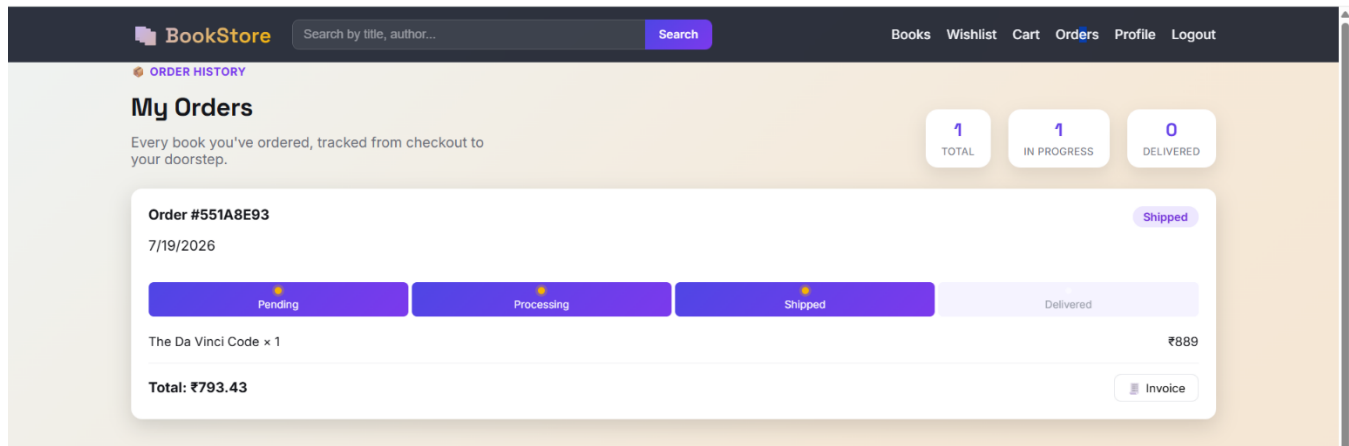
BOOK SECTION:



BookView PAGE:



USER'S ORDER PAGE:



12. Known Issues

- No automated test suite (unit/integration) yet — testing has been manual only
- Payment integration is limited to Cash on Delivery / a placeholder payment method; no real payment gateway is wired in
- Auto-generated book covers are created client-side at the moment of upload; older listings created before this feature will only get a generated cover the next time they are edited
- Email delivery for the forgot-password flow depends on the environment's mail configuration and is not fully exercised in local/dev setup
- No pagination yet on some admin list views (e.g. all reviews) for very large data sets

13. Future Enhancements

- Integrate a real payment gateway (Razorpay/Stripe) alongside Cash on Delivery
- Add automated test coverage (Jest/RTL on the client, Supertest on the server)
- Add real-time order status updates (WebSockets) so customers see fulfillment changes instantly
- Add an AI/ML-based book recommendation engine beyond the current simple recommendation logic
- Support multiple product images per book, and richer seller analytics (sales trends over time)
- Add pagination and bulk actions to remaining admin list views
- Add email/SMS notifications for order status changes and seller approval